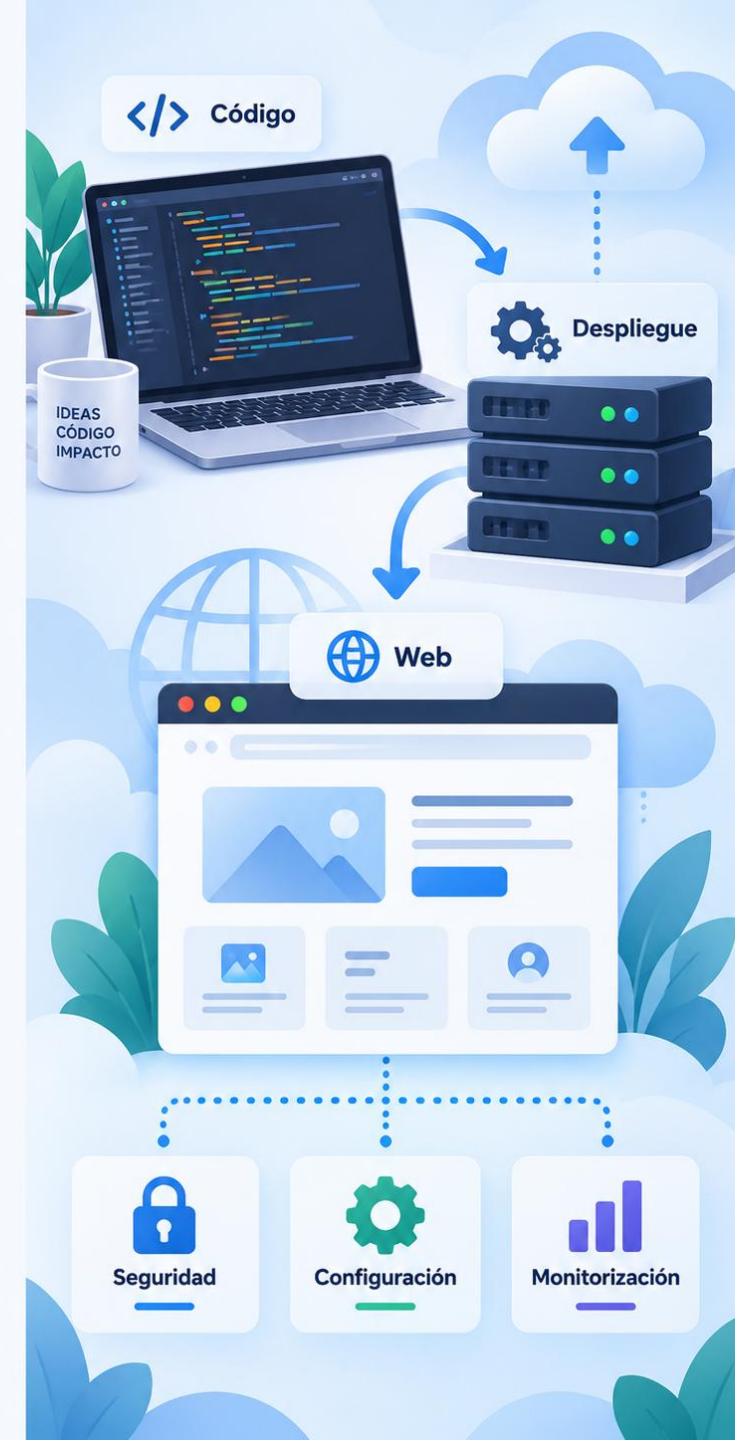


Fundamentos de contenedores

Tema 2 — Virtualización y contenedores



Objetivo de la sesión

Cambiar la pregunta: de “¿funciona aquí?” a “¿puedo empaquetarlo y repetirlo?”

Solo código

- Dependencias fuera del repositorio
- Versiones distintas entre máquinas
- Instalaciones manuales
- Entorno difícil de reconstruir

Unidad reproducible

- Runtime conocido
- Configuración externa
- Entorno aislado
- Misma unidad en distintos entornos

Entorno
reproducible

Imagen

Contenedor

Ejecución

Persistencia

Construcción

Publicación

Idea clave: una imagen empaqueta un entorno; un contenedor es una instancia creada a partir de ella.

Del código al entorno reproducible

Copiar el repositorio no reproduce automáticamente la ejecución.

Qué puede necesitar una aplicación

- Artefacto construido
- Runtime y versiones
- Bibliotecas y herramientas
- Configuración externa
- Servicios auxiliares
- Directorios, permisos y puertos.

Ejemplo: imagen de una aplicación web Java

```
imagen de la aplicación
├── runtime de Java
├── aplicación Spring Boot
└── servidor HTTP embebido
```

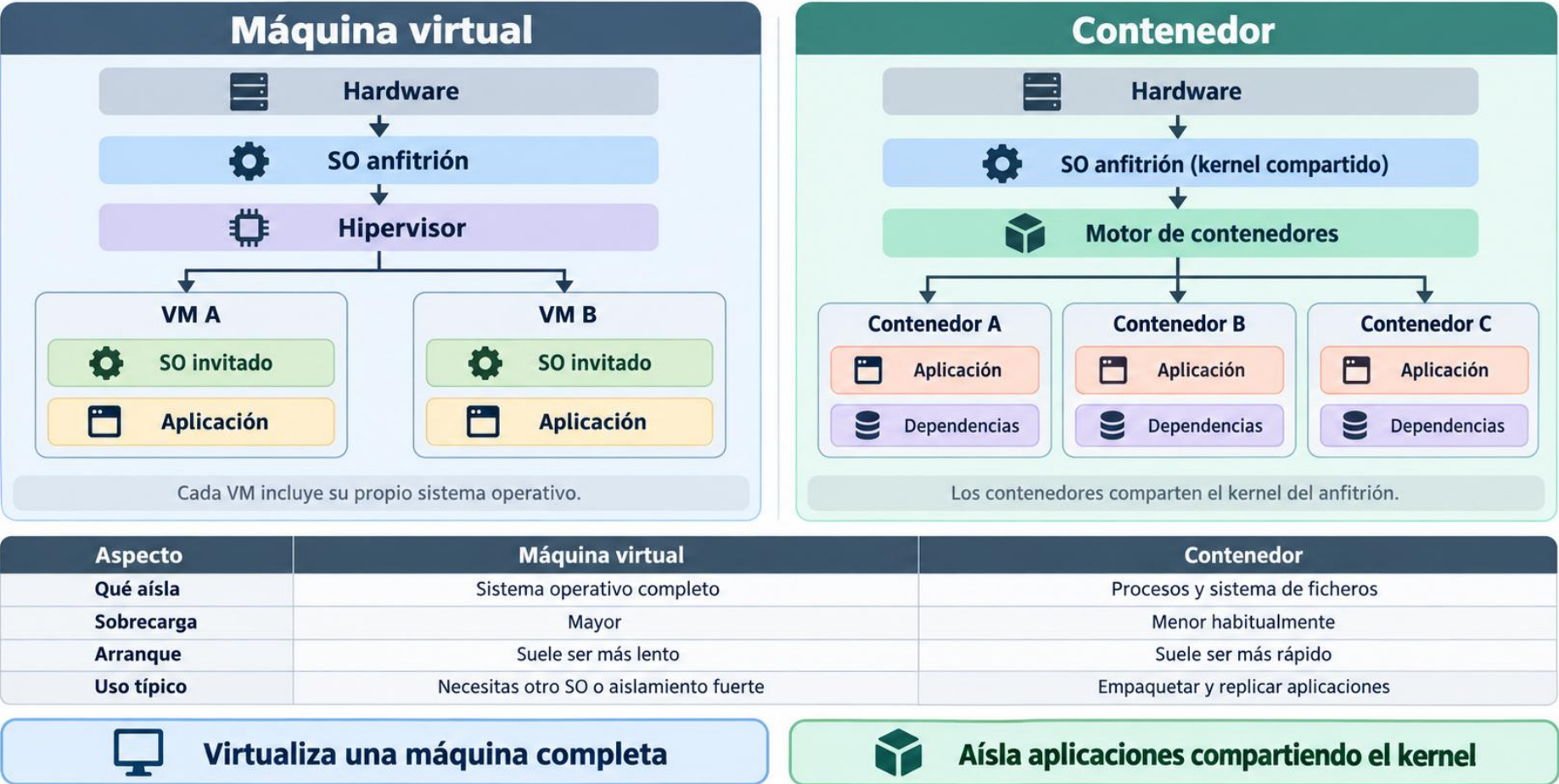
La base de datos, la aplicación y un posible proxy pueden seguir siendo piezas desplegables distintas.

Los contenedores sirven en desarrollo, pruebas, construcción y distribución; no son solo una tecnología de producción.

Máquina virtual y contenedor

Máquina virtual vs contenedor

Dos formas de aislar y ejecutar software

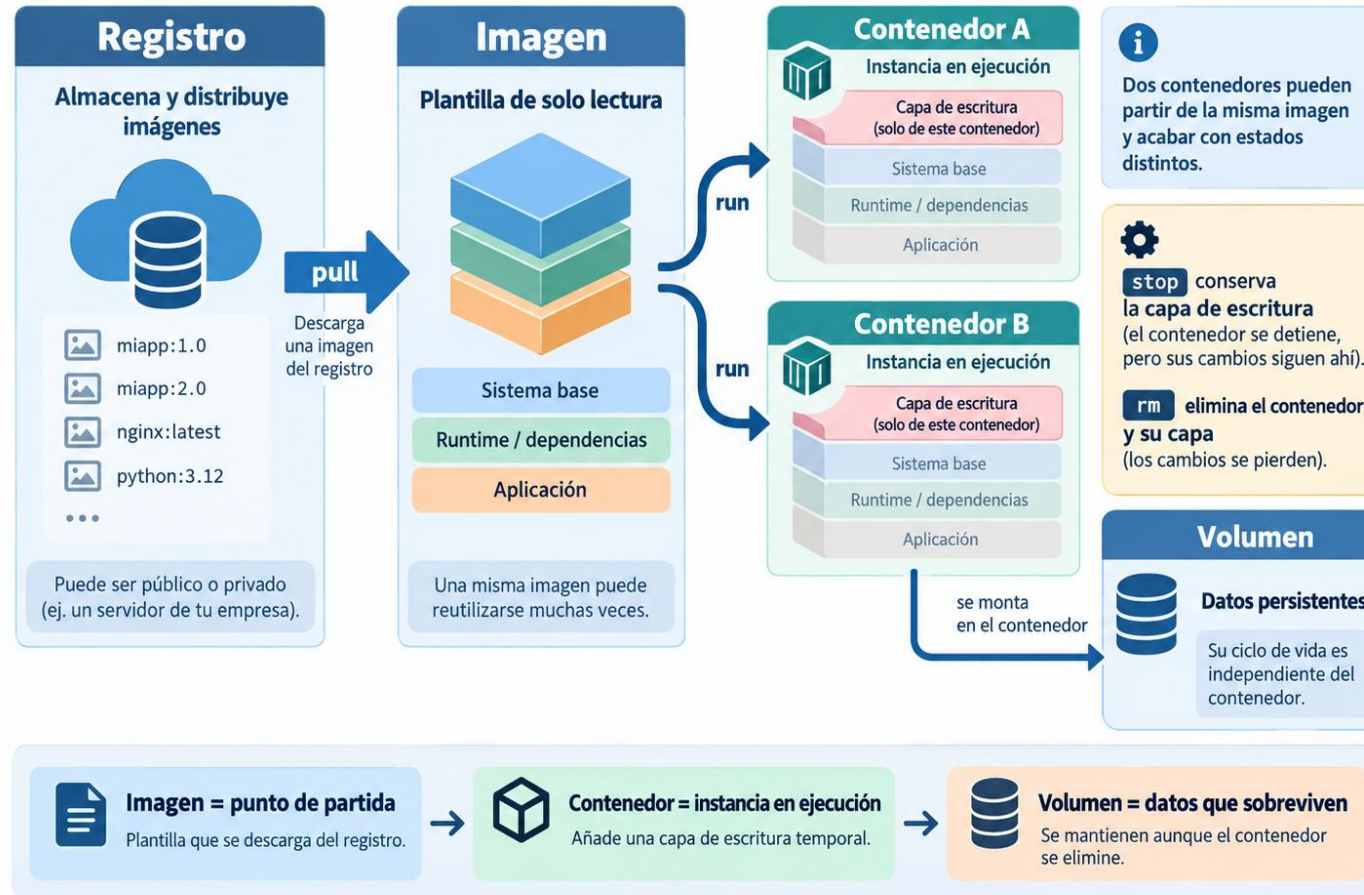


No son alternativas excluyentes: es habitual ejecutar contenedores dentro de máquinas virtuales.

Imagen, contenedor y persistencia

Imagen, contenedor y persistencia

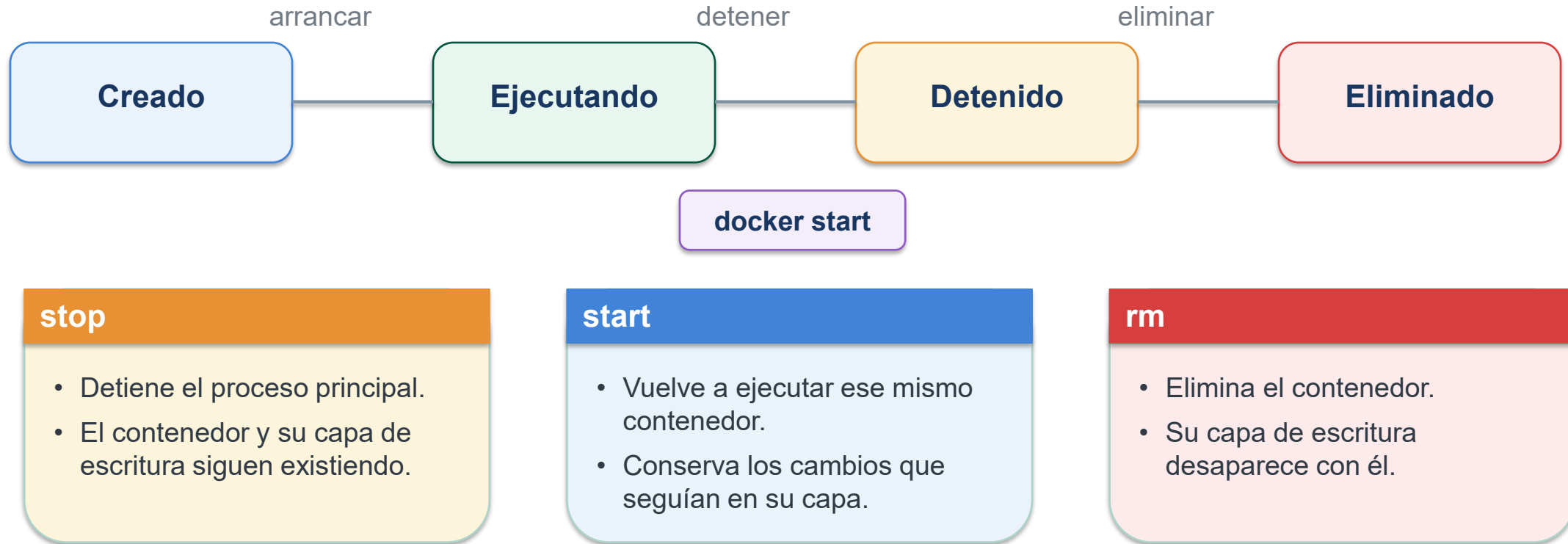
Qué se descarga, qué se ejecuta y qué datos sobreviven



Regla práctica: si un dato debe sobrevivir a `docker rm`, no debería quedarse únicamente en la capa de escritura del contenedor.

Ciclo de vida de un contenedor

Detener un contenedor no es lo mismo que eliminarlo.



Un contenedor en ejecución depende de su proceso principal: si ese proceso termina, el contenedor queda detenido.

Ejecutar, observar y diagnosticar

No memorices comandos: asocia cada uno con una pregunta.

¿Está ejecutándose?

```
docker ps -a
```

¿Qué dicen los logs?

```
docker logs <contenedor>
```

¿Necesito inspeccionarlo?

```
docker exec -it <contenedor> sh
```

¿Con qué configuración arrancó?

```
docker inspect <contenedor>
```

Estado

Logs

Interior

Reflejo útil: estado → logs → interior. No crees contenedores al azar antes de buscar pistas.

Ejecutar, observar y diagnosticar

Ejecutar un contenedor = combinar varias decisiones.

```
docker run -d --name web-demo -p 8088:80 nginx:1.30.4-alpine
```

-d

- Segundo plano
- El terminal queda libre

--name

- Nombre reconocible
- Facilita logs, exec y stop

-p 8088:80

- Host : contenedor
- Publica un puerto al anfitrión

imagen:tag

- Qué se ejecuta
- Versión explícita de la imagen

Comprobaciones inmediatas

```
docker ps
docker logs web-demo
```

Las imágenes mínimas no siempre incluyen bash; para entrar utilizaremos normalmente sh.

Puertos y configuración externa

Publicar un puerto

- Un contenedor no publica automáticamente sus servicios al anfitrión.
- -p anfitrión:contenedor

Ejemplo: navegador → localhost:8080 → puerto 80 del contenedor

```
-p 8080:80
```

Configurar sin construir

- La misma imagen puede arrancar con valores diferentes.
- Las variables se pasan al crear el contenedor.
- Las credenciales reales no deben quedar dentro de la imagen.

```
docker run -d --name bd-demo \  
  -e POSTGRES_DB=tienda_demo \  
  postgres:18-alpine
```

Publicar un puerto y externalizar configuración son decisiones distintas: una abre acceso; la otra adapta la misma imagen al entorno.

Persistencia y comunicación entre contenedores resuelven problemas diferentes.

Persistencia

- **Volumen** → lo gestiona Docker; buen encaje para datos persistentes.
- **Bind mount** → usa una ruta concreta del anfitrión; útil para configuración o ficheros locales.

Si debe sobrevivir al contenedor, debe tener un ciclo de vida independiente.

Redes

- Publicar un puerto → permite acceder desde el anfitrión.
- Red de Docker → permite comunicar contenedores entre sí.
- En redes bridge creadas por el usuario, los servicios pueden descubrirse por nombre.

app → bd

En esta sesión basta con distinguir ambos mecanismos; la configuración explícita llegará en las siguientes prácticas.

Construir una imagen: Dockerfile

Hasta ahora ejecutábamos imágenes de otros; ahora construiremos una propia.

FROM

- Indica la imagen de partida.
- Define el entorno base sobre el que construiremos.

```
FROM alpine:3.24
COPY material/ /datos/
```

COPY

- Incorpora ficheros desde el contexto de construcción.

Dockerfile

docker build

Imagen

docker run

Contenedor

Dockerfile describe cómo construir una imagen; docker run crea un contenedor a partir de esa imagen.

Dockerfile y contexto no son lo mismo

Estructura de ejemplo

```
repositorio/  
├── material/  
│   └── ejemplo.txt  
└── practicas/  
    └── docker/  
        └── demo/  
            └── Dockerfile
```

Construcción desde la raíz del repositorio

```
docker build \  
-f practicas/docker/demo/Dockerfile \  
-t ejemplo:1.0.0 \  
.
```

-f

Dónde está el
Dockerfile

-t

Nombre y etiqueta

.

Contexto que
COPY puede leer

El Dockerfile puede estar en una carpeta y el contexto ser otra. COPY solo puede leer dentro del contexto elegido.

Publicar una imagen

Construir en un lugar y ejecutar en otro.

Imagen local
mi-app:1.0.0

Etiquetar
para GHCR

Autenticar
docker login

Publicar
docker push

Otra máquina
docker pull

Ejecutar
docker run

```
docker tag mi-app:1.0.0 \
ghcr.io/<usuario>/mi-app:1.0.0
```

```
docker login ghcr.io -u <usuario>
docker push ghcr.io/<usuario>/mi-app:1.0.0
```

```
docker pull ghcr.io/<usuario>/mi-app:1.0.0
```

```
docker run ghcr.io/<usuario>/mi-app:1.0.0
```

Publicar separa construcción y ejecución: la máquina destino no necesita tu código fuente ni repetir el build.

Actividad 2.1: primeros contenedores y primera imagen

Aplicarás las ideas de la sesión sobre Nginx, PostgreSQL y el proyecto Escaparate.

Ejecutar

Observar

Modificar

Eliminar y
recrear

Construir
imagen mínima

Publicar
en GHCR

Al salir de la actividad deberías poder...

- Distinguir imagen, contenedor, registro y etiqueta.
- Diagnosticar con estado → logs → interior.
- Explicar qué cambios desaparecen al eliminar un contenedor.
- Construir una imagen mínima eligiendo correctamente el contexto.
- Publicar y volver a descargar una imagen desde GHCR.

La teoría usa ejemplos genéricos; la actividad te obliga a trasladar esas decisiones al proyecto real.

Cierre de la sesión

Lo mínimo que debe quedar claro

- ✓ Una imagen empaqueta un punto de partida reproducible; un contenedor es una instancia concreta.
- ✓ Detener conserva el contenedor; eliminarlo destruye su capa de escritura.
- ✓ Los datos persistentes necesitan un ciclo de vida independiente del contenedor.
- ✓ Publicar un puerto y configurar el contenedor son decisiones explícitas.
- ✓ El Dockerfile describe la construcción y el contexto determina qué puede leer COPY.
- ✓ Un registro permite construir una imagen en un lugar y ejecutarla en otro.

Próxima sesión: construiremos imágenes más eficientes y separaremos claramente build y runtime.